

ASSIGNMENT 6 REPORT - GUI-BASED MULTI-CLIENT CHAT APPLICATION USING TCP

1. OBJECTIVE

Convert the terminal-based chat client into a GUI application while reusing the Assignment 5 server and core socket communication logic.

2. GUI DESIGN

Implemented with tkinter and includes:

- * Login window (username/password/connect)
- * Main chat window with scrollable message area
- * Message input + Send button
- * Disconnect button
- * Online users list panel
- * Connection status label

3. SYSTEM ARCHITECTURE

- * Server: 'server.py' (reused Assignment 5 behavior)
- * GUI Client: 'client_gui.py'
- * Message receive path runs in background thread to keep UI responsive

4. COMPONENTS REUSED FROM ASSIGNMENT 5

- * TCP socket protocol
- * Broadcast messaging
- * Private messaging ('/msg <username> <message>')
- * Online user list update ('ONLINE_USERS:')
- * Join/leave system notifications
- * Persistent chat history format

5. GUI DESIGN DECISIONS

- * Separated network receive loop from tkinter main loop using a background thread.
- * UI updates are marshaled to the GUI thread using 'root.after(...)'.
* Private messaging uses a target username entry and user-list selection helper.

6. TESTING RESULTS

Multi-client tests were executed in Mininet ('single,5') with one server and four clients. Backend communication remained stable while message handling ran concurrently.

7. WIRESHARK VERIFICATION

Capture filter: 'tcp.port == 5000'

Included screenshots:

- * 'screenshots/wireshark_client_connection.png'
- * 'screenshots/wireshark_broadcast_message.png'
- * 'screenshots/wireshark_private_message.png'
- * 'screenshots/wireshark_client_disconnection.png'

These validate connection setup, payload transfer for broadcast/private flows, and disconnection behavior.

8. REFLECTION ANSWERS

1. Networking logic and GUI code should be separated to simplify maintenance, testing, and future upgrades.
2. A background thread is required because blocking socket receives would freeze the GUI event loop.
3. Assignment 5 server routing, command protocol, and online-user/state updates were reused.
4. The GUI improves usability by providing visual message history, online user context, and easier private message targeting.
5. Future enhancement: add authentication and TLS encryption.

9. CONCLUSION

Assignment 6 successfully wraps the prior chat networking stack in a responsive GUI client while preserving multi-client functionality, private messaging, online user awareness, and packet-level verification.